

Old Legacy Machine

28nd July 2025

Prepared By: Hybread

Challenge Author: Hybread

Difficulty: Easy

Writeup classification: Official

Introduction

Old Legacy Machine is an easy cryptography challenge which is quite straightforward. The user is given an 'extracted' hash along with a .exe program that verifies some sort of key. The user must find out what type of hash it is, and then decrypt it by brute forcing, then only will the find the 'secret key' and retrieve the flag from the program.

Description

Years ago, a flag was stored and kept secure by a mini robot created by an engineer. On the workstation, there was a sticky note pasted stating "Password's kinda simple :3". After a long search on the past engineer's super old computer, you and your colleague were able to extract a string of some sort.... it seems familiar BUT it doesn't seem to match what you have in mind. Seems odd.

Skills Required

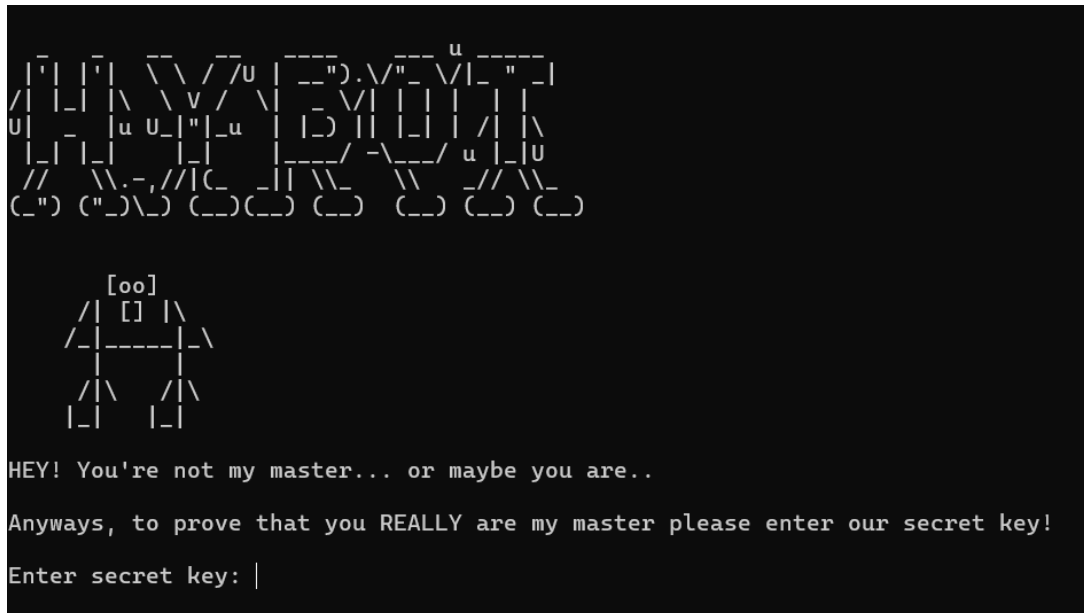
- Basic understanding of cryptography
- Basic knowledge of difference between encryption vs hash
- Knowledge of brute-forcing attack
- Knowledge of hashcat and its hash modes

Skills Learnt

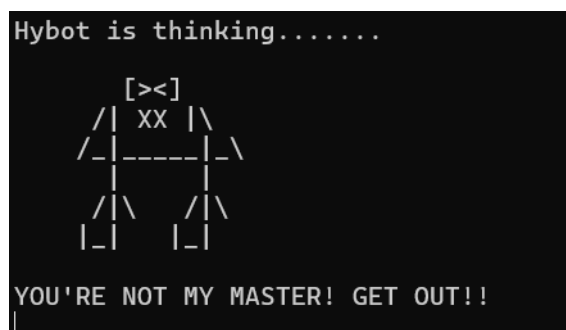
- Learn about NTLM hash type
- Learn how to perform manual hash identification with hashid/hash-identifier
- Learn how to perform a dictionary brute-force attack with hashcat

Solve

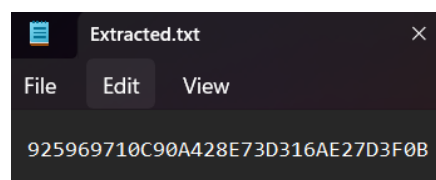
The user will be given a .zip file which contains 3 files, a Hybot.exe program, a list.txt and an extracted.txt file with a string of characters inside. Extract it and when running the program we're greeted with this:



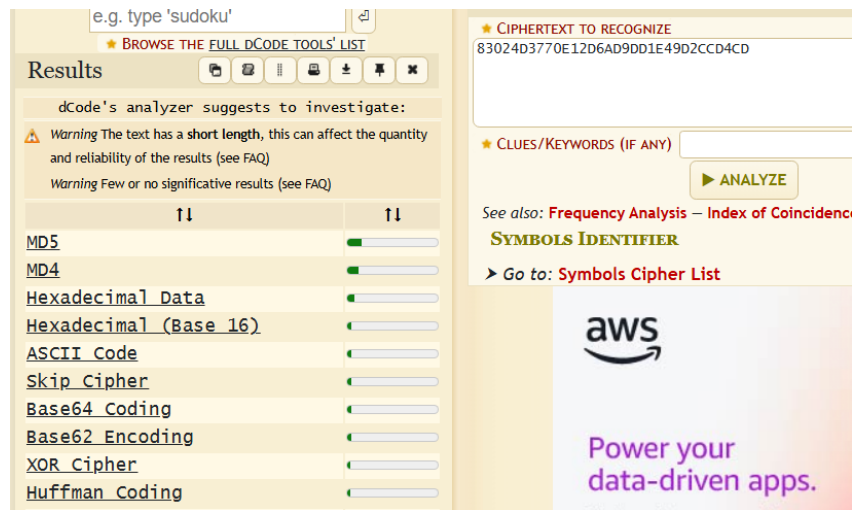
Seems like the program is asking for a 'secret key' or password to be compared before giving us whatever it's storing. Typing an invalid key, the program will reject the input and display an error message:



When the user opens the .txt file, they will be greeted with a hash string:



Visually, it looks as if it's an MD hash string. If the user decides to put it into Dcode's cipher identifier, they will be shown the same thing:



But this isn't the case. If the user attempts to decrypt the hash's original content through the means of online tools such as MD5hashing.net, crackstation.net, etc. They will soon find out that nothing works as those websites are stored wordlists but a familiar theme of brute-forcing.

If the user goes back to the challenge files, they will realize that they must use the provided list.txt which is a modified version of rockyou.txt along with some sort of brute-forcing tool such as hashcat or johntheripper.

To do this, the user must first figure out what type of hash it is. To do so, we'll have to use hashid and/or hash-identifier:

```
(hybread@parrot) ~[~/Desktop]
$ hashid '925969710C90A428E73D316AE27D3F0B'
Analyzing '925969710C90A428E73D316AE27D3F0B'
[+] MD2
[+] MD5
[+] MD4
[+] Double MD5
[+] LM
[+] RIPEMD-128
[+] Haval-128
[+] Tiger-128
[+] Skein-256(128)
[+] Skein-512(128)
[+] Lotus Notes/Domino 5
[+] Skype
[+] Snefru-128
[+] NTLM
[+] Domain Cached Credentials
[+] Domain Cached Credentials 2
[+] DNSSEC(NSEC3)
[+] RAdmin v2.x
```

```
Possible Hashs:
[+] MD5
[+] Domain Cached Credentials - MD4(MD4(($pass)).(strtolower($username)))

Least Possible Hashs:
[+] RAdmin v2.x
[+] NTLM
[+] MD4
[+] MD2
[+] MD5(HMAC)
[+] MD4(HMAC)
```

Here we can see in both results of hashid and hash-identifier, the lesser/least possible hash includes NTLM. NTLM is an old cryptographic representation of a user's password used for authentication in legacy Windows systems. The cause of the results shown is because the way NTLM is generated, matches the format of MD5 and Domain Cached Credentials. A simple google search and matching with the theme & title of the challenge having included 'legacy', the user can figure out that NTLM is the right hash type.

Now that the user knows what type of hash it is, they now need to find out the hash mode for hashcat, with the extra command extension of '-m' in hashid, the user can figure out the proper hash mode to be used:

```
[hybread@parrot]~[~/Desktop]
$ hashid '925969710C90A428E73D316AE27D3F0B' -m
Analyzing '925969710C90A428E73D316AE27D3F0B'
[+] MD2
[+] MD5 [Hashcat Mode: 0]
[+] MD4 [Hashcat Mode: 900]
[+] Double MD5 [Hashcat Mode: 2600]
[+] LM [Hashcat Mode: 3000]
[+] RIPEMD-128
[+] Haval-128
[+] Tiger-128
[+] Skein-256(128)
[+] Skein-512(128)
[+] Lotus Notes/Domino 5 [Hashcat Mode: 8600]
[+] Skype [Hashcat Mode: 23]
[+] Snefru-128
[+] NTLM [Hashcat Mode: 1000]
[+] Domain Cached Credentials [Hashcat Mode: 1100]
[+] Domain Cached Credentials 2 [Hashcat Mode: 2100]
[+] DNSSEC(NSEC3) [Hashcat Mode: 8300]
[+] RAdmin v2.x [Hashcat Mode: 9900]
```

Finally, with everything set in place, the user can use hashcat with this command to extract the original string from the hash:

```
hashcat -a 0 -m 1000 '925969710C90A428E73D316AE27D3F0B' list.txt
```



```
Bytes..... 139921499
* Keyspace...: 14344385
* Runtime...: 4 secs

925969710c90a428e73d316ae27d3f0b: [REDACTED]

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 1000 (NTLM)
Hash.Target.....: 925969710c90a428e73d316ae27d3f0b
Time.Started.....: Mon Jul 28 11:36:18 2025 (1 sec)
Time.Estimated...: Mon Jul 28 11:36:19 2025 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (list.txt)
```

With this the user can take the 'secret key' and enter it into the program and they should get the flag:

```
Hybot is thinking.....
      [^^]
     /  []  \
    /-|-----|- \
    |         |
   /|\      /|\
  | |      | |

WOW, I've missed you master! Here's your flag!
CYNX{ [REDACTED] }
```